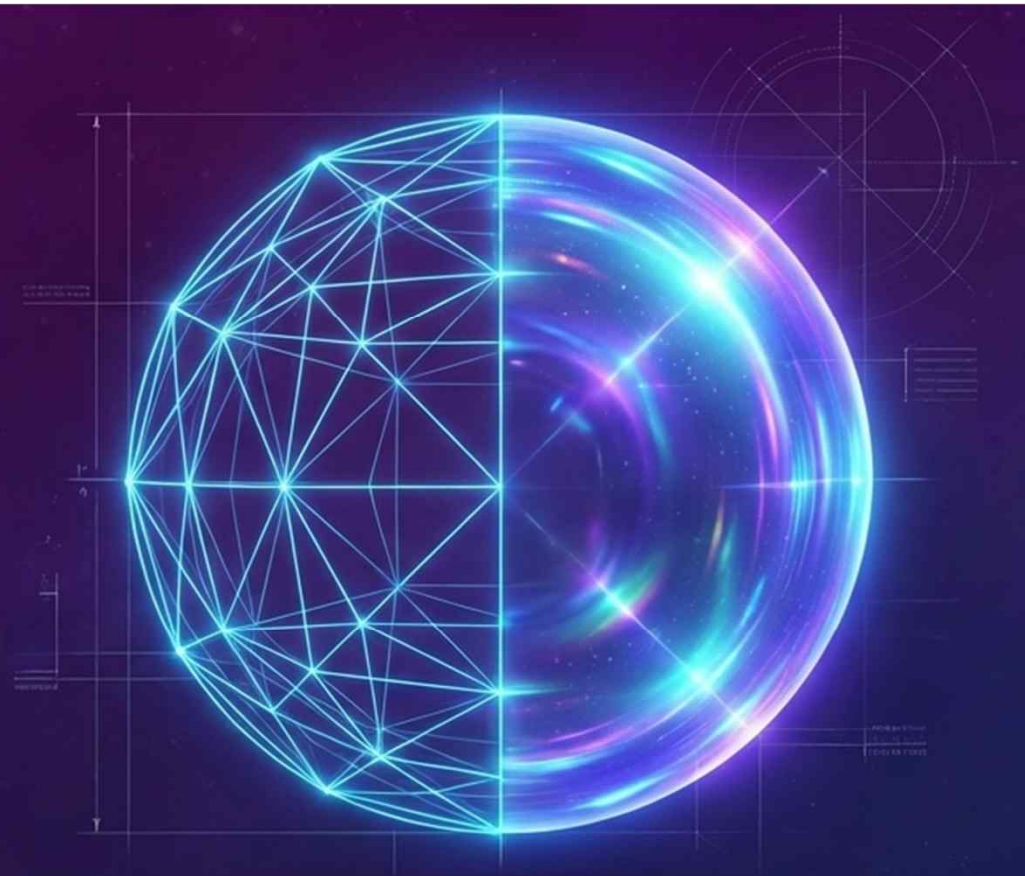


빛의 설계도: Fragment Shader의 이해와 활용

그래픽스 파이프라인의 구조부터 픽셀 제어 실습까지

한국공학대학교
TECH UNIVERSITY OF KOREA

데이터가 화면의 빛이 되기까지의 여정

1단계: 거시적 관점
파이프라인과 Fragment Shader의 위치

2단계: 미시적 관점
래스터화(Rasterization) 및 보간(Interpolation)

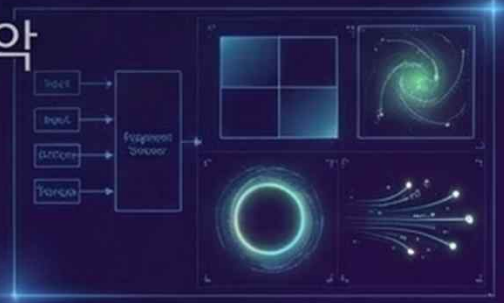
3단계: 도구 준비
FS의 입력과 저장 한정자 (Storage Qualifiers)

4단계: 실전 구축
화면 분할, 원형 렌더링, 파티클 세이밍 실습

01

파이프라인과 Fragment Shader

화면에 그려질 픽셀의 운명을 결정하는
최종 렌더링 스테이지의 위치 파악

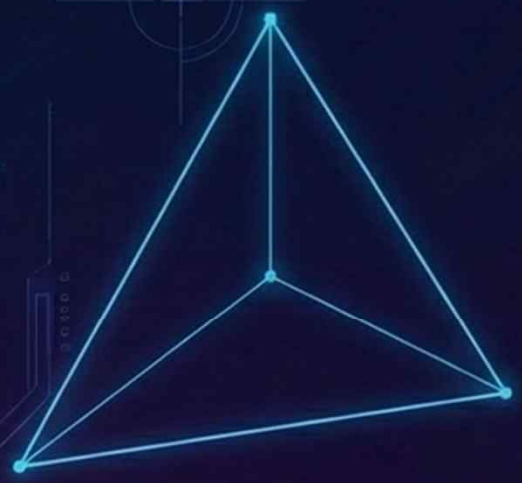


OpenGL ES 3.0 그래픽스 파이프라인

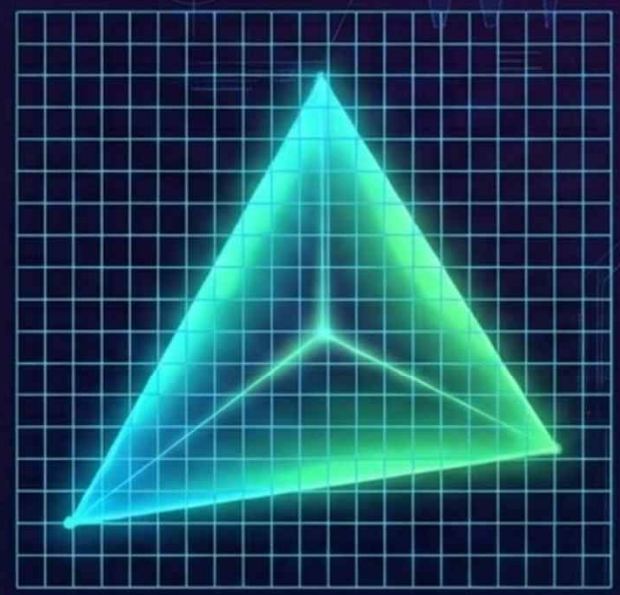
파이프라인의 전반부는 '기하학(Geometry)'을 다루고, 후반부는 '픽셀(Pixel)'을 다룹니다.

한국공학대학교
TECH UNIVERSITY OF KOREA

Fragment Shader의 본질적 역할



각 프래그먼트(픽셀 후보)마다
한 번씩 독립적으로 실행됩니다.

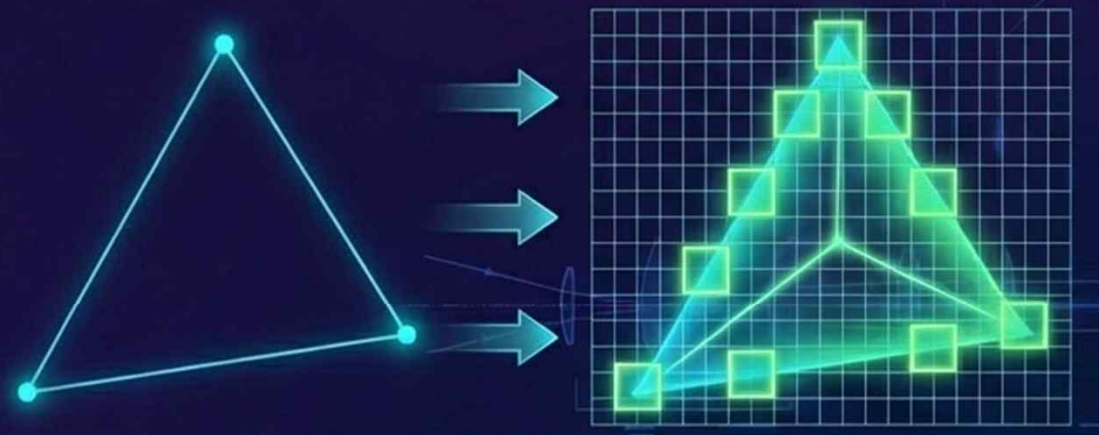


조명, 텍스처, 그림자, 색상 연산을
수행하여 최종 픽셀의 색상(Color)과
깊이(Depth)를 결정합니다.

한국공학대학교
TECH UNIVERSITY OF KOREA

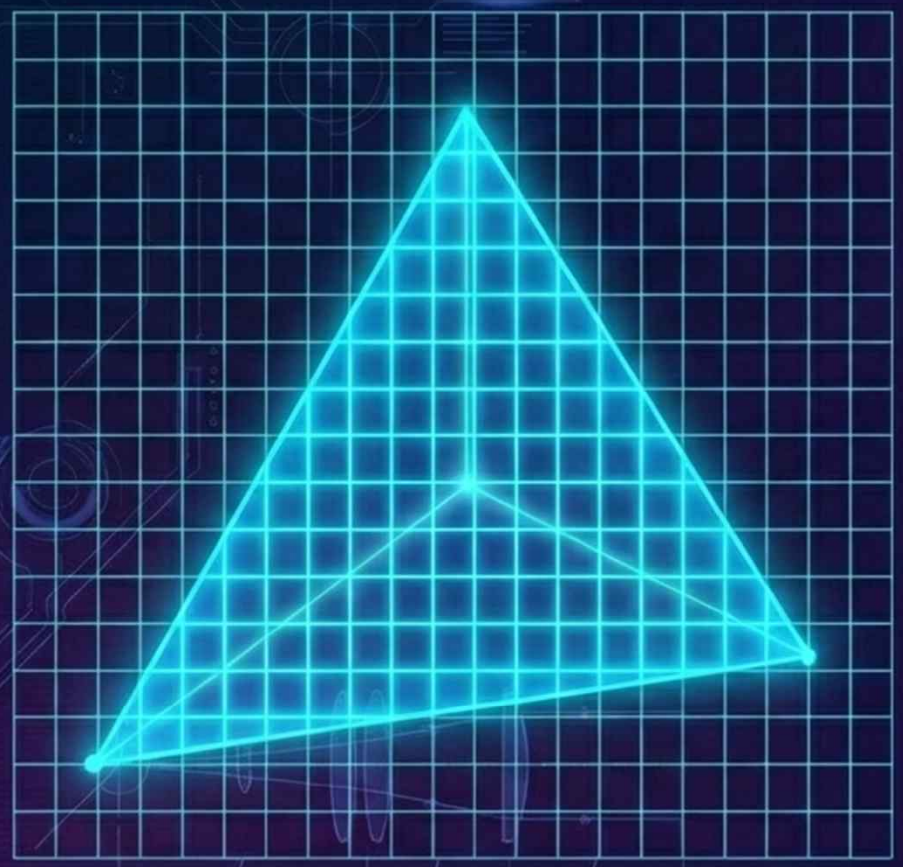
02 래스터화와 보간

점과 선은 어떻게 면(프래그먼트)이 되는가?



한국공학대학교
TECH UNIVERSITY OF KOREA

래스터화 (Rasterization): 벡터에서 격자로



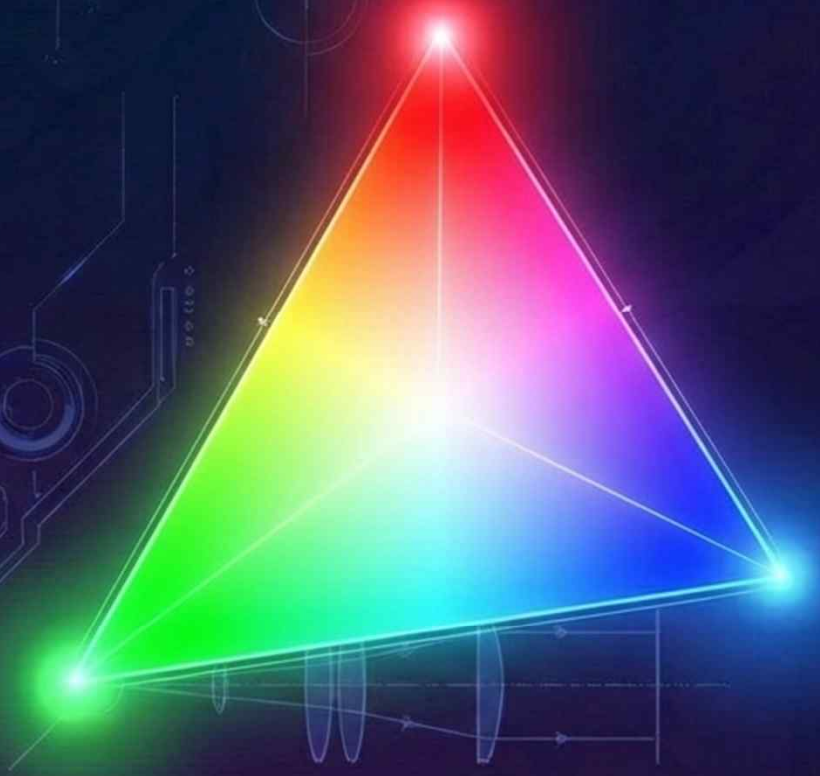
정의: 기하학적 도형(Primitive)을
2차원 화면의 픽셀 그리드에 매핑하는 과정.

결과물: 화면의 픽셀과 1:1로 대응하는
'프래그먼트(Fragment)'들의 집합을 생성합니다.

핵심: 이 단계에서 생성된 프래그먼트의 수만큼
Fragment Shader가 독립적으로 호출됩니다.

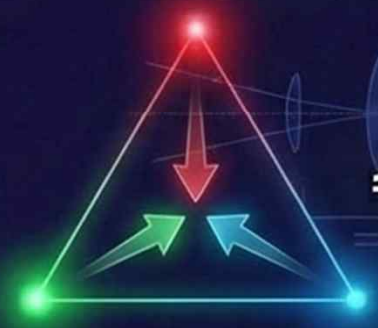
한국공학대학교
TECH UNIVERSITY OF KOREA

보간 (Interpolation): 빈 공간을 채우는 수학



자동화된 데이터 채움

- **정의:** 정점(Vertex) 사이의 값을 거리 비율에 따라 수학적으로 계산하여 채워 넣는 과정.
- **의의:** Vertex Shader에서 단 3번 출력된 색상 값이 보간을 거쳐, 삼각형 내부의 수천 개 프래그먼트에 고유한 입력값으로 전달됩니다.



수학적 가중치 혼합

한국공학대학교
TECH UNIVERSITY OF KOREA

03

Fragment Shader의 데이터 재료들

파이프라인 단계 간, 그리고 CPU와 GPU 간의 데이터 통신 규약

Storage Qualifier (저장 한정자) 종합 비교

한정자 (Qualifier)	접근 권한 (Access)	데이터 출처 (Source)	특징 (Characteristics)
in	읽기 전용 (Read-only)	이전 스테이지 (VS)	래스터화/보간을 거쳐 복사되어 들어온 입력값
out	쓰기 전용 (Write-only)	현재 스테이지 (FS)	다음 스테이지(Frame Buffer)로 내보낼 출력값
uniform	읽기 전용 (Read-only)	외부 어플리케이션 (CPU)	모든 프래그먼트에 동일하게 적용되는 전역 변수 (예: Time, MVP Matrix)
const	읽기 전용 (Read-only)	셰이더 내부 (Local)	컴파일 시 결정되어 변경할 수 없는 상수

스테이지 간의 연결고리: in & out

Vertex Shader

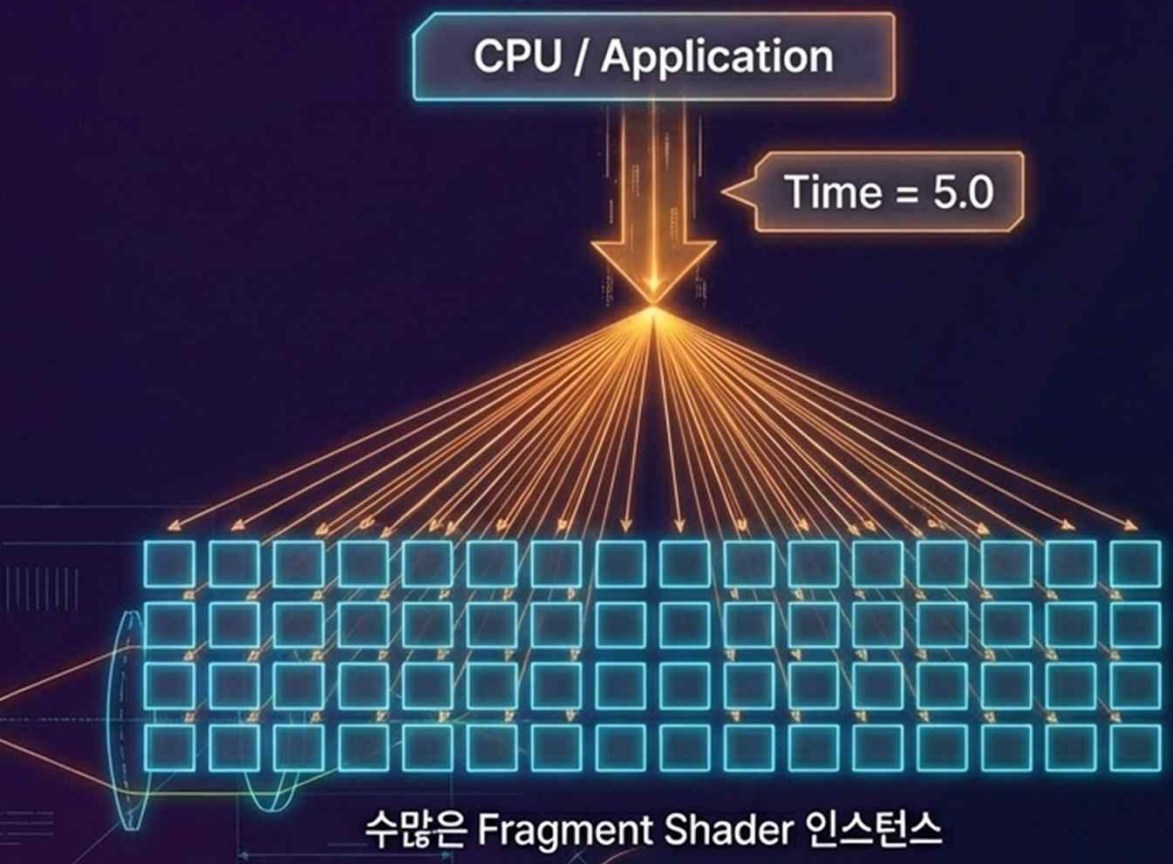
```
// Vertex Shader
out vec4 vColor; // 다음 스테이지로 전달

void main() {
    vColor = vec4(1.0, 0.0, 0.0, 1.0);
}
```

Fragment Shader

```
// Fragment Shader
in vec4 vColor;  // 이전 스테이지에서 수신 (보간됨)
out vec4 outColor; // 화면으로 출력

void main() {
    outColor = vColor;
}
```

주의: 이름과 타입이 정확히 일치해야 파이프라인이 데이터를 올바르게 연결(Linkage)할 수 있습니다.

외부로부터의 공통 지시: uniform



uniform은 렌더링되는 단일 Primitive 내에서 값이 변하지 않는 전역 데이터입니다.
(예: 애니메이션 시간, 변환 행렬, 전반적인 색상 톤)

Next Step

텍스처 매핑(Texture Mapping) 예고:
이미지 데이터를 픽셀에 입히는 작업 역시 'uniform sampler2D'를 통해 외부 이미지 데이터를 받아 처리합니다.
(다음 심화 강의에서 상세히 다룹니다).

한국공학대학교 TECH UNIVERSITY OF KOREA

04

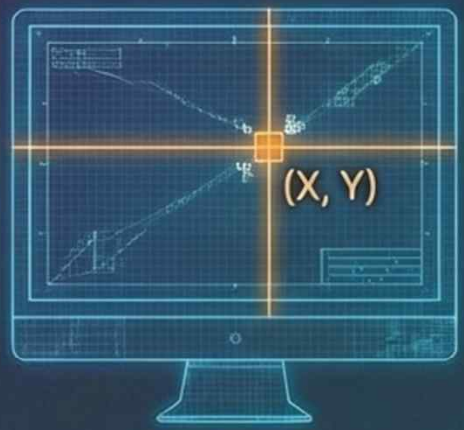
실습 1. 화면 영역 분할하기

내장 변수와 조건 제어문을 이용한 기초 프래그먼트 조작

좌표를 읽고 화면을 쪼개다

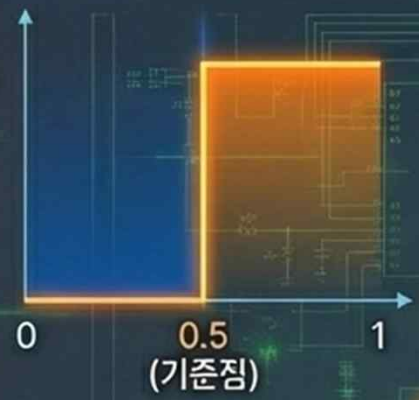
개념 1: gl_FragCoord

- Fragment Shader에서 기본적으로 제공하는 내장 변수(Built-in variable).
- 현재 처리 중인 프래그먼트의 화면 픽셀 좌표(X, Y)를 담고 있습니다.



개념 2: 분할 논리 (step() 또는 if)

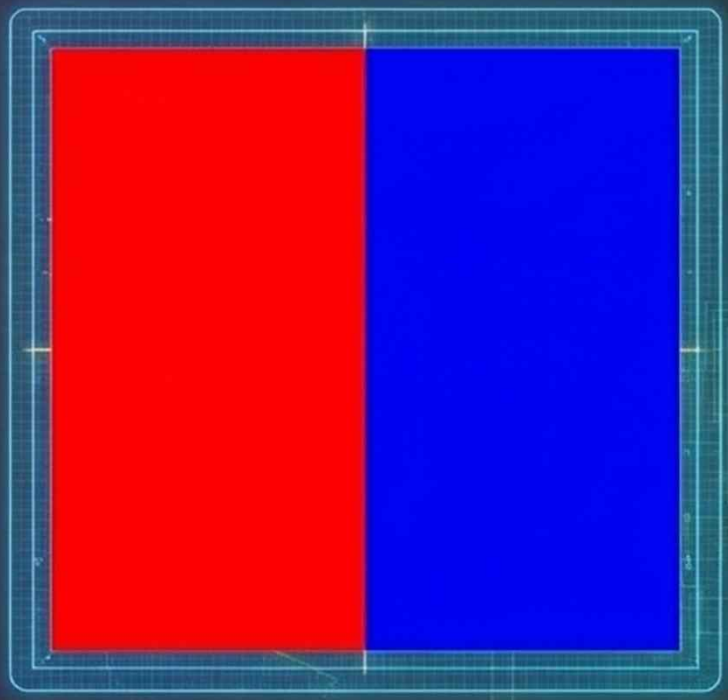
- 화면의 특정 기준점(예: 폭의 50%)을 넘었는지 판단하여 다른 색상을 출력합니다.
- Tip: GPU 아키텍처에서는 분기문(if)보다 내장 수학 함수(step())를 사용하는 것이 성능에 유리한 경우가 많습니다.



한국공학대학교
TECH UNIVERSITY OF KOREA

실습 1 결과: 좌우 분할 렌더링

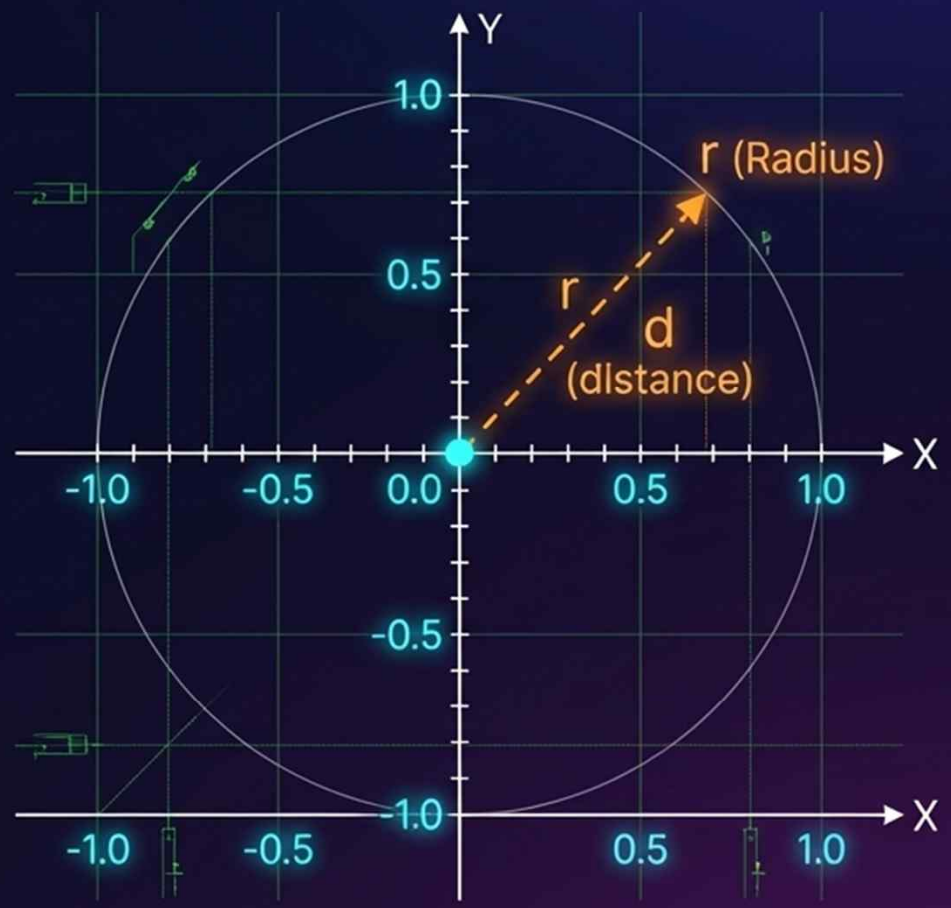
```
in vec2 vUV; // 0.0 ~ 1.0으로 정규화된 좌표
out vec4 fragColor;

void main() {
    // x좌표가 0.5보다 크면 1.0, 작으면 0.0 반환
    float split = step(0.5, vUV.x);

    // split 값에 따라 붉은색과 푸른색 혼합
    vec3 color = mix(vec3(1.0, 0.0, 0.0),
                     vec3(0.0, 0.0, 1.0),
                     split);

    fragColor = vec4(color, 1.0);
}
```



05 실습 2. 수학으로 동그라미 그리기

거리 함수(Distance Function)와 프래그먼트 폐기(Discard)

원의 방정식과 거리(Distance) 계산

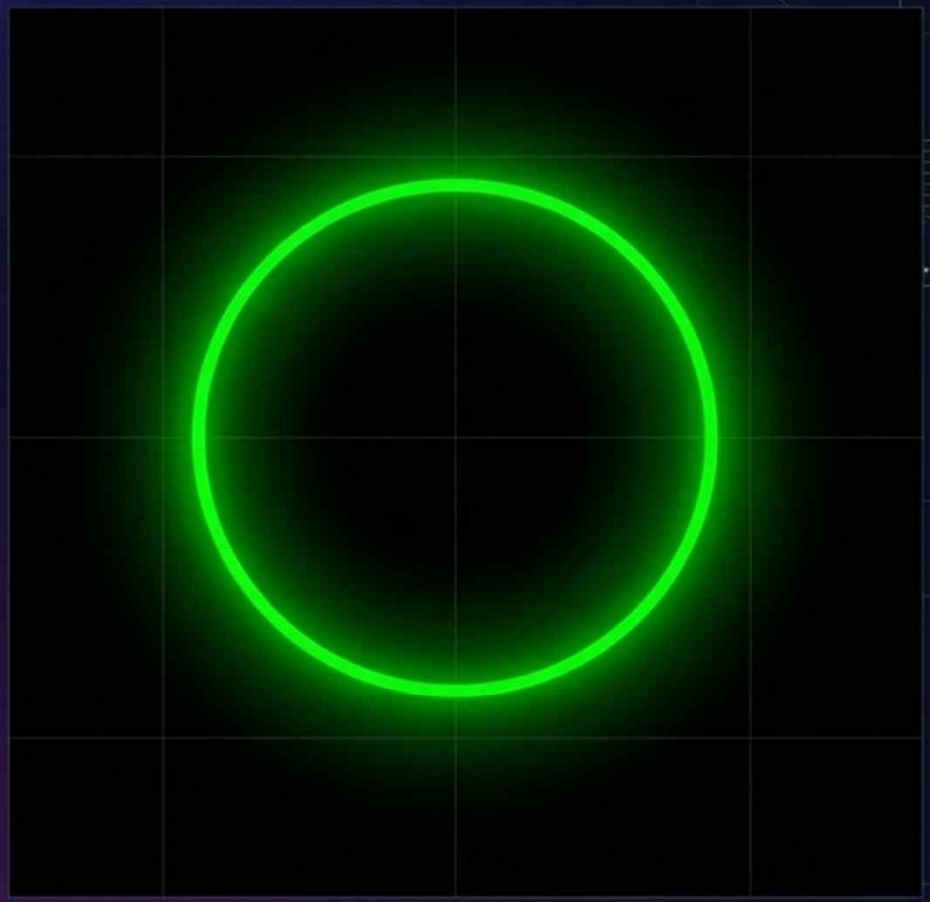


- 화면 중심점(Center)으로부터 현재 프래그먼트 좌표(UV)까지의 거리를 계산합니다.
- GLSL 내장 함수 length() 또는 distance()를 활용하여 거리를 구합니다.
- 계산된 거리가 지정한 반지름(Radius)보다 크면 색상을 칠하지 않고 버립니다(discard 키워드 사용). 작으면 색상을 채웁니다.

한국공학대학교
TECH UNIVERSITY OF KOREA

실습 2 결과: 거리 기반 원형 렌더링

```
in vec2 vUV; // -1.0 ~ 1.0 범위
out vec4 fragColor;

void main() {
  // 중심(0,0)에서 현재 픽셀까지의 거리
  float dist = length(vUV);

  // 반지름이 0.5를 넘어가면 픽셀 렌더링 취소
  if(dist > 0.5) {
    discard;
  }

  fragColor = vec4(0.0, 1.0, 0.0, 1.0);
  // 네온 그린
}
```

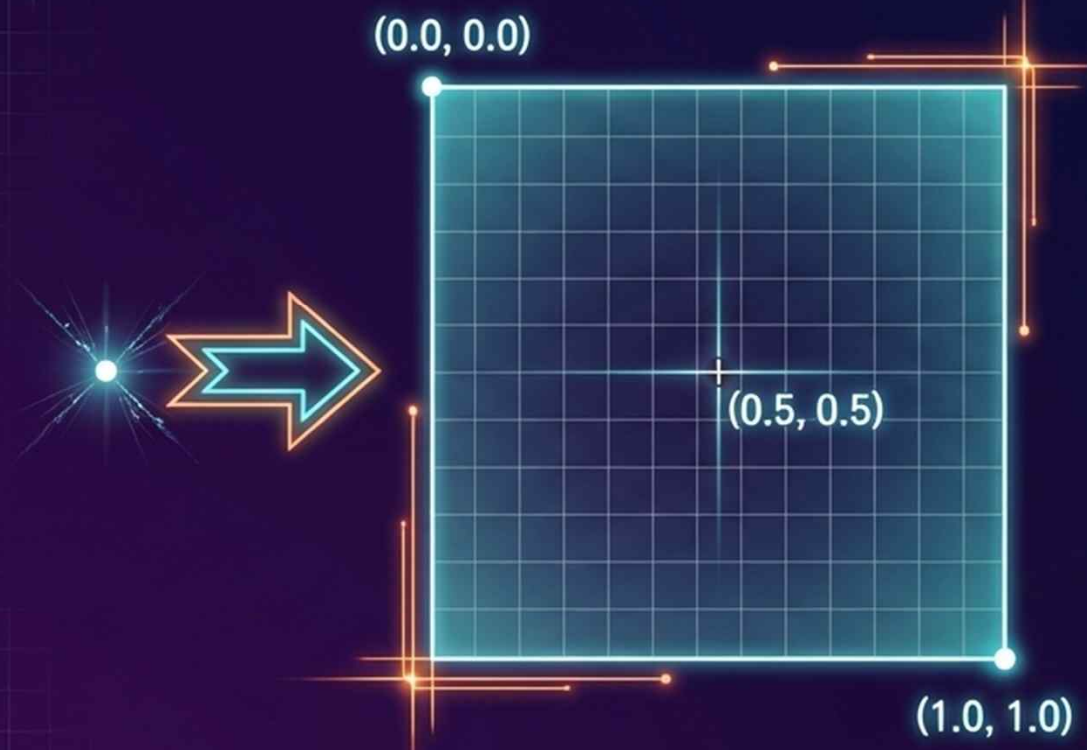


06

실습 3. 파티클 모양을 동그랗게 다듬기

Point Sprite와 내장 텍스처 좌표계의 활용

Point Sprite와 gl_PointCoord



문제점: 점(GL_POINTS) 데이터를 크게 렌더링하면 기본적으로 딱딱한 '정사각형' 모양으로 그려집니다.

해결책: gl_PointCoord 사용

점 렌더링 시 Fragment Shader 내부적으로 자동 생성되는 0.0 ~ 1.0 범위의 자체 텍스처 좌표계입니다.

이 내부 좌표계를 중심점(0.5, 0.5)을 기준으로 삼아 앞서 배운 거리 함수를 적용하면, 거친 사각형 파티클을 부드럽고 둥글게 깎아낼 수 있습니다.

한국공학대학교
TECH UNIVERSITY OF KOREA

실습 3 결과 및 요약: 둥근 파티클의 완성

```
out vec4 fragColor;
void main() {
    // gl_PointCoord는 중심이 (0.5, 0.5)
    vec2 centerCoord = gl_PointCoord - vec2(0.5);

    if(length(centerCoord) > 0.5) {
        discard;
    }

    fragColor = vec4(1.0, 0.5, 0.0, 1.0); // 오렌지색 파티클
}
```

강의 요약: Fragment Shader는 보간된 입력값(in)과 외부 제어값(uniform)을 바탕으로, 래스터화된 수많은 프래그먼트의 운명(색상과 생존 여부)을 픽셀 단위의 수학으로 정밀하게 설계하는 진정한 '빛의 설계도'입니다.